

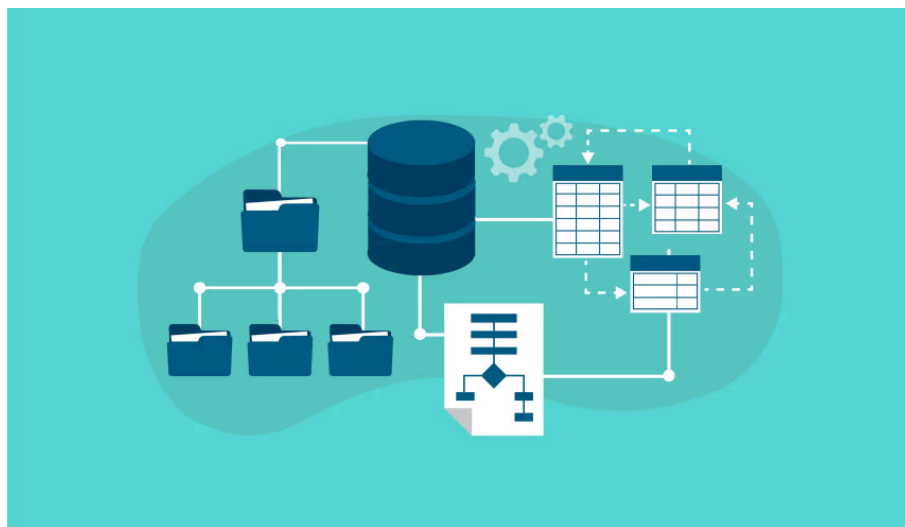
Actividad 4

Materia: Estructura de Datos

Maestro: Adalberto Emmanuel Rojas Perea

Alumna: Isabel Kyra Romero Borunda

TECMILENIO



Índice

Índice.....	2
Introducción.....	3
Implementación.....	3
Clases.....	3
Nodo.....	3
ArbolBinario.....	3
Operaciones.....	3
Resultados.....	6
ÁRBOL.....	6
Agregar.....	6
Recorridos.....	7
Eliminación.....	7
Busqueda.....	8
EJEMPLO EMPLEADOS.....	8
Ver Empleado.....	8
Insertar Empleado.....	8
Busqueda Empleados.....	9
Conclusión.....	9

Introducción

En esta actividad se emplea el concepto de un árbol binario para hacer una simulación que consta de un sistema que nos permite agregar, ver y buscar empleados. Donde estos empleados cuentan con un ID y su nombre.

Nota:

Se usó como guía el código proporcionado para esta actividad.

Implementación

Clases

Nodo

El nodo contiene 4 atributos donde 'valor' representa el ID, 'nombre' es donde se guarda el nombre del empleado, 'hijoizquierdo' y 'hijoderecho' representan las hojas del árbol.

```
class Nodo {
    int valor;
    String nombre;
    Nodo hijoizquierdo, hijoderecho;
    public Nodo(int valor, String nombre) {
        this.valor = valor;
        this.nombre = nombre;
        this.hijoizquierdo = null;
        this.hijoderecho = null;
    }
}
```

ArbolBinario

Operaciones

- **Agregar**

Este método de agregar es para un árbol binario de búsqueda(BST), aquí los nodos se van agregando de tal manera que todos los nodos a la izquierda son menores a los nodo de la derecha.

```
// Insertar nodo
public void agregarNodo(int valor, String nom) {
    Nodo nuevo = new Nodo(valor, nom);
    if (raiz == null) {
        raiz = nuevo;
        return;
    }
    Nodo auxiliar = raiz;
    Nodo padre;
    while (true) {
        padre = auxiliar;
        if (valor < auxiliar.valor) {
            auxiliar = auxiliar.hijoizquierdo;
            if (auxiliar == null) {
                padre.hijoizquierdo = nuevo;
                return;
            }
        } else {
            auxiliar = auxiliar.hijoderecho;
            if (auxiliar == null) {
                padre.hijoderecho = nuevo;
                return;
            }
        }
    }
}
```

- Eliminar

Se utiliza la recursividad en este método. Aquí hay 3 casos a la hora de eliminar una hoja, caso 1 donde si el nodo es **una hoja**, simplemente lo eliminamos y devolvemos 'null'. Caso 2 donde si el nodo tiene **solo un hijo**, el nodo a eliminar se reemplaza por su hijo. Luego el caso 3, donde si el nodo tiene **dos hijos**, se busca el sucesor en inorden (el nodo más pequeño del subárbol derecho) después se copia el valor del sucesor al nodo que quiero eliminar y por último se elimina el sucesor.

```
// Eliminación
public Nodo eliminar(Nodo nodo, int valor) {
    if (nodo == null) return null;

    if (valor < nodo.valor) {
        nodo.hijoizquierdo = eliminar(nodo.hijoizquierdo, valor);
    } else if (valor > nodo.valor) {
        nodo.hijoderecho = eliminar(nodo.hijoderecho, valor);
    } else {
        // Caso 1: sin hijos
        if (nodo.hijoizquierdo == null && nodo.hijoderecho == null) {
            return null;
        }
        // Caso 2: un solo hijo
        else if (nodo.hijoizquierdo == null) {
            return nodo.hijoderecho;
        } else if (nodo.hijoderecho == null) {
            return nodo.hijoizquierdo;
        }
        // Caso 3: dos hijos
        else {
            Nodo sucesor = encontrarMinimo(nodo.hijoderecho);
            nodo.valor = sucesor.valor;
            nodo.nombre = sucesor.nombre;
            nodo.hijoderecho = eliminar(nodo.hijoderecho, sucesor.valor);
        }
    }
    return nodo;
}
```

- Encontrar minimo

Este método sirve para encontrar el nodo con el valor más pequeño en un subárbol de un árbol binario de búsqueda. La relación que tiene con el método eliminar se encuentra en que al momento de eliminar un nodo con dos hijos, este reemplaza lo reemplaza con el sucesor en inorden(mínimo del subárbol derecho).

```
// Encontrar el nodo con valor mínimo en un subárbol
private Nodo encontrarMinimo(Nodo nodo) {
    while (nodo.hijoizquierdo != null) {
        nodo = nodo.hijoizquierdo;
    }
    return nodo;
}
```

- **Buscar**

Aquí también usamos la recursividad, este método busca el nodo con el valor(ID) de este, si el nodo es 'null' nos regresa 'null'. Si el valor del nodo es igual al ID nos regresa ese nodo.

```
// Buscar nodo
public Nodo buscar(Nodo nodo, int id) {
    if (nodo == null)
        return null;

    if (nodo.valor == id) {
        return nodo;
    }

    Nodo izq = buscar(nodo.hijoizquierdo, id);
    if (izq != null) return izq;

    return buscar(nodo.hijoderecho, id);
}
```

- **Mostrar**

Este método nos imprime el ID junto con el nombre del empleado.

```
// Mostrar arbol
public void mostrar(Nodo nodo, String espacio) {
    if(nodo == null) return;
    System.out.println(espacio + "ID: " + nodo.valor + " -> Nombre: " + nodo.nombre);
    mostrar(nodo.hijoizquierdo, espacio);
    mostrar(nodo.hijoderecho, espacio);
}
```

- **InOrde, PreOrden y PostOrden**

Estos métodos recorren el árbol de distintas maneras.

```
// Inorden
public void inorden() {
    inorden(raiz);
}
private void inorden(Nodo nodo) {
    if (nodo != null) {
        inorden(nodo.hijoizquierdo);
        System.out.println(nodo.valor + " -> " + nodo.nombre);
        inorden(nodo.hijoderecho);
    }
}
```

```
// Preorden
public void preorden() {
    preorden(raiz);
}
private void preorden(Nodo nodo) {
    if (nodo != null) {
        System.out.println(nodo.valor + " -> " + nodo.nombre);
        preorden(nodo.hijoizquierdo);
        preorden(nodo.hijoderecho);
    }
}
```

```
// Postorden
public void postorden() {
    postorden(raiz);
}
private void postorden(Nodo nodo) {
    if (nodo != null) {
        postorden(nodo.hijoizquierdo);
        postorden(nodo.hijoderecho);
        System.out.println(nodo.valor + " -> " + nodo.nombre);
    }
}
```

- Esta Vacio

Nos dice si el árbol está vacío.

```
public boolean estaVacio() {
    return raiz == null;
}
```

Resultados

ÁRBOL

Agregar

Primero ingresamos por el código nodos con su valor y un dato string.

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    ArbolBinario arbol = new ArbolBinario();

    // Agregar nodo
    arbol.agregarNodo(valor:40, nom:"Raíz");
    arbol.agregarNodo(valor:20, nom:"Izq");
    arbol.agregarNodo(valor:10, nom:"Der");
    arbol.agregarNodo(valor:50, nom:"Izq-Izq");
    arbol.agregarNodo(valor:30, nom:"Izq-Der");
    arbol.agregarNodo(valor:60, nom:"Der-Izq");
    arbol.agregarNodo(valor:70, nom:"Der-Der");
}
```

Luego utilizamos el método 'mostrar' para que no imprima todos los nodos que agregamos al árbol.

```
// Mostrar
System.out.println(x: "\n");
arbol.mostrar(arbol.raiz, espacio: "");
```

Podemos ver en la imagen de al lado cómo se imprimen en la terminal.

```
ID: 40 -> Nombre: Raíz
ID: 20 -> Nombre: Izq
ID: 10 -> Nombre: Der
ID: 30 -> Nombre: Izq-Der
ID: 50 -> Nombre: Izq-Izq
ID: 60 -> Nombre: Der-Izq
ID: 70 -> Nombre: Der-Der
```

Recorridos

Utilizamos los métodos 'preorden', 'inorden' y 'postorden'. Podemos ver en las imágenes como se imprimen los recorridos.

```
// Recorridos
System.out.println(x: "\nPreOrden");
arbol.preorden();

System.out.println(x: "\nInOrden");
arbol.inorden();

System.out.println(x: "\nPostOrden");
arbol.postorden();
```

```
PreOrden
40 -> Raíz
20 -> Izq
10 -> Der
30 -> Izq-Der
50 -> Izq-Izq
60 -> Der-Izq
70 -> Der-Der
```

```
InOrden
10 -> Der
20 -> Izq
30 -> Izq-Der
40 -> Raíz
50 -> Izq-Izq
60 -> Der-Izq
70 -> Der-Der
```

```
InOrden
10 -> Der
20 -> Izq
30 -> Izq-Der
40 -> Raíz
50 -> Izq-Izq
60 -> Der-Izq
70 -> Der-Der
```

Eliminación

Utilizamos el método 'eliminar' y luego el método 'mostrar' para verificar que estén si se hayan eliminado. Podemos ver en la imagen a la derecha que primero se imprime el árbol sin haber eliminado y luego se imprime de nuevo con las eliminaciones hechas.

```
// Eliminacion
arbol.eliminar(arbol.raiz, valor:70);
arbol.eliminar(arbol.raiz, valor:10);
arbol.eliminar(arbol.raiz, valor:40);

System.out.println(x: "\n");
arbol.mostrar(arbol.raiz, espacio: "");
```

```
ID: 40 -> Nombre: Raíz
ID: 20 -> Nombre: Izq
ID: 10 -> Nombre: Der
ID: 30 -> Nombre: Izq-Der
ID: 50 -> Nombre: Izq-Izq
ID: 60 -> Nombre: Der-Izq
ID: 70 -> Nombre: Der-Der
```

```
ID: 50 -> Nombre: Izq-Izq
ID: 20 -> Nombre: Izq
ID: 30 -> Nombre: Izq-Der
ID: 60 -> Nombre: Der-Izq
```

Busqueda

Utilizamos el método 'buscar' y luego hacemos que imprima el dato guardado del nodo que se busco de acuerdo a su valor numérico.

```
// Busqueda
System.out.println(x:"\n");
Nodo buscar = arbol.buscar(arbol.raiz, id:30);
Nodo buscar2 = arbol.buscar(arbol.raiz, id:60);
System.out.println("Buscando Nodo con valor de 30: \nNodo encontrado: " + buscar.nombre);
System.out.println("Buscando Nodo con valor de 60: \nNodo encontrado: " + buscar2.nombre);
```

Aquí se ve que imprime los datos de los nodos buscados.

```
Buscando Nodo con valor de 30:
Nodo encontrado: Izq-Der
Buscando Nodo con valor de 60:
Nodo encontrado: Der-Izq
```

EJEMPLO EMPLEADOS

Primero se agregaron a unos empleados desde el código:

```
// ID: Contiene 6 números, esta compuesto por el año,el mes y el numero de empleado registrado
int numEmpleado = 3;
arbol.agregarNodo(valor:101001, nom:"Isabel");
arbol.agregarNodo(valor:100802, nom:"Dafne");
arbol.agregarNodo(valor:100903, nom:"Jocelyn");
```

Recordatorio:

El ID se constituye de 6 números, los primero dos dígitos es el año en el que se contrató al empleado, luego los dos dígitos de en medio representa el mes y los últimos 2 es el número de empleado.

Ver Empleado

Elegimos la opción 1 para ver a los empleados actuales, donde los imprime con el ID y el nombre de este.

```
1-. Ver empleados
2-. Insertar empleado
3-. Buscar Empleado por ID
4-. Salir
>>> 1

EMPLEADOS
ID: 101001 -> Nombre: Isabel
ID: 100802 -> Nombre: Dafne
ID: 100903 -> Nombre: Jocelyn
```

Insertar Empleado

Aquí seleccionamos la opción 2 para insertar a un empleado, en este caso insertamos a 2 empleados.

```
1-. Ver empleados
2-. Insertar empleado
3-. Buscar Empleado por ID
4-. Salir
>>> 2
```

```
Ingrese el ID(año, mes, número actual de empleado: 4) y el nombre del Empleado:
>>> ID: 250104
>>> Nombre: Isabella
```

```
Ingrese el ID(año, mes, número actual de empleado: 5) y el nombre del Empleado:
>>> ID: 250205
>>> Nombre: Valentin
```

Al final imprimimos a los empleados para ver que se hayan agregado correctamente.

```
1-. Ver empleados
2-. Insertar empleado
3-. Buscar Empleado por ID
4-. Salir
>>> 1

EMPLEADOS
ID: 101001 -> Nombre: Isabel
ID: 100802 -> Nombre: Dafne
ID: 100903 -> Nombre: Jocelyn
ID: 250104 -> Nombre: Isabella
ID: 250205 -> Nombre: Valentin
```

Busqueda Empleados

Seleccionamos la opción 3 para buscar a un empleado por su ID, ingresamos el ID del empleado y luego nos imprime el nombre de este. Se buscaron a dos empleados como ejemplo.

```
1-. Ver empleados
2-. Insertar empleado
3-. Buscar Empleado por ID
4-. Salir
>>> 3
```

```
Ingrese el ID(año, mes, número actual de empleado: 5) del empleadp que quiere buscar:
>>> ID: 250104
Empleado encontrado:
ID: 250104
Nombre: Isabella
```

```
Ingrese el ID(año, mes, número actual de empleado: 5) del empleadp que quiere buscar:
>>> ID: 100903
Empleado encontrado:
ID: 100903
Nombre: Jocelyn
```

Conclusión

El sistema de árbol binario mejora la eficiencia, porque al momento de recorrer todos los empleados (búsqueda secuencial), reduce el problema dividiéndolo en mitades a cada paso.

Debido a esto es importante, ya que reduciendo el problema también reduce la utilización de recursos.